

*** moonBASIC ***

The Completely Unofficial, Mildly Sweary, Actually Useful Guide to moonBASIC

*Or: How to Make Games Without Selling Your Soul
to a 47-Button Inspector Panel*

A fun, mildly sweary guide to the language, the engine, and shipping vibes — validated against real moonBASIC docs and samples.

Expanded edition · CharmingBlaze · github.com/CharmingBlaze/moonbasic

Or: How to Make Games Without Selling Your Soul to a 47-Button Inspector Panel

Expanded edition (v2 — chunky). 50 chapters, deep-dive walkthroughs, genre recipes, a week-long curriculum, contributor appendix, and more ways to open a window and accidentally invent a genre.

***Disclaimer:** Loud friendly voice. Occasional swearing. Language facts match this engine. Official dry docs still live under `docs/` if you need the courtroom transcript.*

***Audience:** Absolute beginners, Blitz refugees, Unity survivors, and anyone who typed `Graphics 640, 480` once in 2004 and never emotionally recovered.*

***How to use this book:** Top to bottom, or panic-skip. Samples point at real files (`examples/...`). Steal those. Stealing sample code is a sacred tradition.*

***Edition note:** When this book and a reference page disagree on a parameter, trust `docs/API_CONSISTENCY.md / compiler/builtinmanifest/commands.json`. This book is vibes + velocity.*

Table of Contents

Part I — Welcome to the Moon

1. What the hell is moonBASIC?
2. Install without losing your mind
3. moonbasic vs moonrun (the #1 footgun)
4. Your first hour (a guided panic)
5. Projects, folders, and moonbasic new

Part II — Speak the Language

6. Variables, types, and why # is dead
7. Arrays, TYPE, NEW, DELETE
8. Control flow that doesn't hate you
9. Functions, callbacks, coroutines
10. INCLUDE, IMPORT, and splitting your brain
11. Style: how cool kids write moonBASIC

Part III — The Sacred Game Loop

12. Open a window or go home
13. Input: keys, mouse, pads, ACTIONS
14. Delta time, cameras 2D, and not teleporting
15. DRAW.* — rectangles, text, HUD glory

16. Your first 2D game: Pong energy

Part IV — Into the Third Dimension

17. Cameras, cubes, and existential dread

18. Entities: your toys with handles

19. DRAW3D, grids, and debug geometry

20. Models, meshes, materials, textures

21. Lights, fog, and "why is everything black"

Part V — Physics: Newton's Revenge

22. Jolt, gravity, and respectful bouncing

23. Characters that walk like humans (mostly)

24. Collision without a full physics world

25. 2D physics with Box2D

Part VI — Make an Actual Game

26. Platformers, hoppers, Mario-ish vibes

27. Annotated tour: Star Road showcase

28. Sprites, tilemaps, and the 2D renaissance

29. Audio that doesn't ghost you

30. GUI menus that don't softlock

31. Particles and juice

32. Math helpers so you stop reinventing SQRT

33. Saves, files, JSON

Part VII — Bigger Than Your Bedroom

34. Assets pipeline and packs

35. Terrain, open worlds, atmosphere

36. Multiplayer: when friends become lag

37. Easy Mode for Blitz dinosaurs

38. Memory, FREE, and not leaking the moon

Part VIII — Shipping & Surviving

39. Debug like a detective

40. Performance without becoming a monk

41. Ship it to other humans

42. Where to go next

Part IX — Deep Cuts (the "I want more" section)

43. Full walkthrough: build Pong in your head

44. Full walkthrough: manual 2D platformer

45. Game recipes (copy-paste genres)

46. Strings, printing, and talking to humans

47. A week-long curriculum

48. FAQ from the trenches

49. For engine contributors (optional pain)

50. Porting from Blitz / DBPro without crying

Appendices

- A. Mega cheat sheet
 - B. Namespace phone book
 - C. Common "why is it broken" list
 - D. Map of the official docs
 - E. Glossary
 - F. Sample catalog (steal these)
 - G. Key constants you'll actually use
 - H. "Add systems in this order" poster
-

Part I — Welcome to the Moon

Chapter 1 — What the hell is moonBASIC?

moonBASIC is a **modern BASIC for games**.

Not "BASIC" like your uncle's calculator homework from 1987. More like: *BlitzBASIC had a baby with a real compiler, then that baby stole Raylib's graphics wallet, Jolt's physics gym membership, Box2D's 2D side hustle, and ENet's networking Rolodex.*

You write `.mb` files. A compiler turns them into `.mbc` bytecode. A VM runs that bytecode. The runtime opens windows, draws stuff, simulates physics, plays audio, and occasionally makes you feel like a genius.

The pitch, without the marketing cologne

- **One language** for 2D and 3D
- **Readable as hell** — `IF hp < 20 THEN` still means what you think
- **Real engine guts** — not a toy REPL with dreams
- **Same Path** — write once, run on Windows and Linux without rewriting your soul every Tuesday
- **Namespaces** — `WINDOW.OPEN`, `ENTITY.SETPOS`, `AUDIO.PLAY` — find shit by *what you're trying to do*
- **Vertical stack** — compiler + VM + engine in one toolchain

What it is *not*

- Not Unity with a BASIC cosplay
- Not "learn 14 design patterns before your first rectangle"
- Not Blitz with the `#` / `$` / `%` punctuation parade (those suffixes are **gone** — celebrate)
- Not a hidden `Game.Loop()` that steals control — **your WHILE stays visible on purpose**

The mental model in one breath

```
.mb source → compiler → .mbc bytecode → VM → HAL (Null or Raylib) → pixels / audio / physics
```

End users live on the left: edit `.mb`, hit `moonrun`, smile or scream, repeat.

Contributors also care about **fullruntime**, CGO, and Jolt build tags. Beginners: ignore that until chapter 41's cousin (the appendix of pain).

Architecture for curious cats

Layer	Job
Lexer / parser / semantic / codegen	Turn <code>.mb</code> into IR / bytecode
VM	Execute <code>.mbc</code>
Runtime modules	<code>ENTITY</code> , <code>PHYSICS3D</code> , <code>AUDIO</code> , ...
HAL (<code>rt.Driver</code>)	Null (headless) or Raylib (windowed)

Headless exists so `--check` and CI don't need a GPU. Your game does.

A tiny taste

```
APP.OPEN(800, 600, "Hello damn moon") APP.SETFPS(60) WHILE NOT  
APP.SHOULDCLOSE() RENDER.CLEAR(20, 24, 40) DRAW.TEXT("I made a window. I am  
unstoppable.", 40, 40, 22, 255, 255, 255, 255) RENDER.FRAME() WEND APP.CLOSE()
```

Complete interactive program. No `MonoBehaviour`. No `ManagerManager`. You're already dangerous.

Chapter 2 — Install without losing your mind

Windows first. Linux second. That's the house rule — this book follows it.

Path A: IDE bundle (recommended)

1. [GitHub Releases](#)
2. Download **IDE bundle**
 - Windows: `moonbasic-<tag>-ide-windows-amd64.zip`
 - Linux: `moonbasic-<tag>-ide-linux-amd64.tar.gz`
 - macOS Apple Silicon: `...-ide-macos-arm64.tar.gz`
3. Extract somewhere **permanent** (not Downloads)
4. **START-IDE.bat** / **START-IDE.command** / **./START-IDE.sh**
5. Open `samples/hello.mb` → **F5**

Docs are built into the IDE. Optional: **ADD-TO-PATH** so terminals find `moonbasic` / `moonrun`.

Path B: Full runtime (terminal)

Same Releases page → **full runtime** zip/tarball. You need both:

- `moonbasic` — check, compile, LSP, `new`
- `moonrun` — **opens a window and plays**

```
moonbasic new MyFirstGame cd MyFirstGame moonrun main.mb
```

Windows: `moonrun.exe` if not on PATH. Linux: may need Mesa / Wayland / X11 packages (OS graphics stack — not Raylib sidecars).

Path C: Compiler-only (wrong bag of holding for games)

Has `moonbasic`, **no `moonrun`**. CI loves it. Cubes hate it.

VS Code / Cursor

```
moonbasic install-vscode
```

Or double-click `INSTALL-VSCODE.bat` / `run ./INSTALL-VSCODE.sh`. Gets syntax, LSP, tasks, and debug via `moonrun --dap` when full runtime is present.

Debug tip: open a `.mb`, set breakpoints, **Run and Debug** → **Debug moonBASIC**. Globals scope shows live variables. Feels illegal. Is legal.

You do not need Go to make games

Building from source is a contributor hobby (`go build -tags fullruntime ...`). Players use releases. If someone demands MinGW before your first `PRINT`, they're contributing to the engine — or trolling.

Keep release tags married

Ship `.mbc` with a **matching** full-runtime version. Mixing random tags is how you invent mysterious crashes.

Chapter 3 — moonbasic vs moonrun (the #1 footgun)

Tool	What it does	Window?
<code>moonrun game.mb</code>	Compile (if needed) + play	Yes
<code>moonbasic --check game.mb</code>	Lint / validate	No
<code>moonbasic game.mb</code>	Write <code>game.mbc</code>	No

<code>moonbasic new Name</code>	Scaffold project	No
<code>moonbasic --lsp</code>	Language server	No
<code>moonbasic package windows</code>	Packaging helper (see ship chapter)	No
<code>HELP("WINDOW.OPEN")</code>	In-game / script help text	Depends

The tragedy

```
# Does NOT play: moonbasic main.mb # DOES: moonrun main.mb
```

Also fine: `moonrun game.mbc` after you've compiled.

Contributor footgun

```
go run -tags fullruntime ./cmd/moonrun path/to/game.mb
```

Without `fullruntime` you're in Null-driver land. Cool for tests. Useless for vibes.

File extensions

Ext	Meaning
<code>.mb</code>	Source
<code>.mbc</code>	Bytecode (magic <code>MOON</code> , see <code>ARCHITECTURE.md</code>)

Day-to-day: edit `.mb`, run with `moonrun`.

Chapter 4 — Your first hour (a guided panic)

Official cousin: `docs/FIRST_HOUR.md` and `docs/BEGIN_HERE.md`.

Minute 0–5: exist

```
moonbasic new MyFirstGame cd MyFirstGame moonrun main.mb
```

A window appears. Change the title string. Run again. Dopamine.

Minute 5–15: move something in 2D

```
APP.OPEN(800, 450, "Mover") APP.SETFPS(60) x = 400.0 y = 225.0 WHILE NOT  
(INPUT.KEYDOWN(KEY_ESCAPE) OR APP.SHOULDCLOSE()) dt = APP.DELTA() x = x +  
INPUT.AXIS(KEY_A, KEY_D) * 220 * dt y = y + INPUT.AXIS(KEY_W, KEY_S) * 220 *  
dt RENDER.CLEAR(18, 20, 28) DRAW.CIRCLE(INT(x), INT(y), 18, 255, 180, 80, 255)  
DRAW.TEXT("WASD move ESC quit", 16, 12, 18, 230, 230, 240, 255) RENDER.FRAME()  
WEND APP.CLOSE()
```

Minute 15–40: spin a cube

```
moonrun examples/spin_cube/main.mb
```

Then open that file and change `.col(255, 150, 70)` to something hideous. Taste is optional. Learning is not.

Minute 40–60: hop

```
moonrun examples/mario64/modern_blitz_hop.mb
```

WASD, orbit camera, space jump. If physics feels stubby on your build, read Part V — Jolt wants full runtime + CGO on Windows/Linux.

Suggested add-on order (from `docs/systems/00-START.md`)

1. Loop that presents frames
2. `DEBUG` / HUD numbers
3. `INPUT` / `ACTION`
4. Assets
5. Physics
6. Audio
7. Save
8. Network (later — much later)

Chapter 5 — Projects, folders, and `moonbasic new`

```
moonbasic new MyGame # optional templates (when available): # moonbasic new  
MyGame --template 3d # moonbasic new MyGame --template platformer
```

Typical scaffold:

```
MyGame/ main.mb assets/ .vscode/ ; launch, tasks, extensions README.md
```

Habits that scale

- Put art/audio under `assets/` and load relative paths from the project root
- `INCLUDE "player.mb"` when `main.mb` becomes a novel
- `moonbasic --check main.mb` before you blame the GPU
- Keep a `notes.txt` of controls — future you is an idiot who forgot the jump key

Workflow loop

```
edit → moonbasic --check → moonrun → swear → edit
```

That's the whole job.

Part II — Speak the Language

Chapter 6 — Variables, types, and why # is dead

Assign and chill

```
name = "Hero" hp = 100 speed = 5.5 alive = TRUE
```

Dynamically typed. Inferred on assignment. You *can* put a string where a number lived — the compiler may shrug; your game may not.

No Blitz suffixes

Blitz	moonBASIC
speed#	speed
msg\$	msg
ok?	ok
x%	x

Case doesn't care (you should)

```
score = SCORE. Window.Open = WINDOW.OPEN.
```

Style: camelCase vars, SCREAMING_SNAKE consts, PascalCase types, uppercase namespaces in docs.

Scope

```
FUNCTION Boom() LOCAL message = "local only" STATIC callCount = 0 callCount = callCount + 1 ENDFUNCTION
```

Globals by default. LOCAL / STATIC / GLOBAL inside functions.

Constants and enums

```
CONST MAX_HP = 100 ENUM State IDLE WALK RUN JUMP ENDENUM IF playerState = State.IDLE THEN PRINT "vibing" ; also: STATE_IDLE
```

String interpolation

```
PRINT($"Score: {score} Health: {hp}") PRINT($"Yaw: {yaw:.2f}")
```

{expr} → STR. {expr:fmt} → FORMAT. Also STRING.INTERP("Hi {0}", name) for positional style.

Don't shadow namespaces

```
time = 3 ; may punch TIME.* input = 1 ; may punch INPUT.* window = 0 ; you see  
where this is going
```

Name clocks elapsed, pads pad0, windows winw. Stay friends with the standard library.

Chapter 7 — Arrays, TYPE, NEW, DELETE

DIM arrays

```
DIM enemies(10) ; often 1..n style in demos DIM scores(5) scores(1) = 100
```

arr.length is handy. ERASE scores frees that array.

Typed records

```
TYPE Platform x, y, z w, h, d r, g, b ENDTYPE ; FIELD spelling also works  
(Blitz comfort food): TYPE Enemy FIELD mesh FIELD body FIELD hp ENDTYPE CONST  
N = 4 DIM plat AS Platform(N) ; indices 0 .. N-1 in LANGUAGE.md examples  
plat(0) = Platform(0.0, 1.5, 6.0, 4.0, 0.4, 4.0, 255, 60, 200) PRINT plat(0).x  
plat(0).r = 200
```

Heads-up: some samples use **1-based** typed arrays by habit (plats(1) in Star Road). Know which index world you're in before you invent an off-by-one religion.

FOR EACH vs EACH(Type)

```
; Walk a DIM array FOR EACH e IN enemies IF e.hp <= 0 THEN ENTITY.FREE(e.mesh)  
NEXT ; Walk live NEW instances e = NEW(Enemy) e.hp = 100 FOR e = EACH(Enemy)  
IF e.hp <= 0 THEN DELETE e NEXT
```

NEW / DELETE manage heap instances the VM tracks. Typed DIM is a fixed slot array. Different tools.

Nuclear option

```
ERASE ALL ; or FREE.ALL
```

Frees **VM heap objects** and nulls handle-holding globals. Does **not** replace `ENTITY.CLEARSCENE` / `ENTITY.FREE` for numeric entity IDs. Don't use mid-expression. Don't name a variable `ALL`.

Chapter 8 — Control flow that doesn't hate you

IF / SELECT

```
IF score > 1000 THEN PRINT "High score!" ELSEIF score > 500 THEN PRINT
"Respectable" ELSE PRINT "Touch grass" ENDIF SELECT fruit CASE "apple" PRINT
"Doctor deferred" CASE "banana" PRINT "Potassium" DEFAULT PRINT "Mysterious
produce" ENDSELECT
```

Loop buffet

```
FOR i = 1 TO 10 PRINT i NEXT FOR i = 10 TO 1 STEP -2 PRINT i NEXT x = 0 WHILE
x < 5 x = x + 1 WEND REPEAT PRINT "at least once" UNTIL TRUE DO WHILE x < 10 x
= x + 1 LOOP DO x = x - 1 LOOP UNTIL x = 0
```

Early out: `EXIT FOR` / `EXIT WHILE` / `EXIT REPEAT` / `EXIT DO`.

The **NOT** trap (tattoo #2)

```
; WRONG — NOT binds tight: WHILE NOT INPUT.KEYDOWN(KEY_ESCAPE) OR
APP.SHOULDCLOSE() ; RIGHT: WHILE NOT (INPUT.KEYDOWN(KEY_ESCAPE) OR
APP.SHOULDCLOSE())
```

Parentheses: cheaper than therapy.

Chapter 9 — Functions, callbacks, coroutines

Basics + multi-return

```
FUNCTION Add(a, b) RETURN a + b ENDFUNCTION FUNCTION GetPlayerPos() RETURN px,
py, pz ENDFUNCTION x, y, z = GetPlayerPos()
```

Optional types:

```
FUNCTION Add(a AS FLOAT, b AS FLOAT) AS FLOAT RETURN a + b ENDFUNCTION
```

`EXIT FUNCTION` for early bail.

Function references & anonymous funcs

```
FUNCTION OnHit(a, b) PRINT "hit!" ENDFUNCTION PHYSICS3D.ONCOLLISION(bodyA,
bodyB, @OnHit) onHit = FUNCTION(a, b) PRINT "hit!" ENDFUNCTION cb = @OnHit
cb(bodyA, bodyB)
```

Legacy string callback names still exist on some APIs. Prefer `@Name` so typos fail earlier.

Coroutines (cutscenes, patrols, drama)

```
COROUTINE patrol WHILE TRUE PRINT "step" COROUTINE.WAIT(1.0) YIELD WEND
ENDCOROUTINE ; or explicit: FUNCTION Patrol() WHILE TRUE COROUTINE.WAIT(1.0)
YIELD WEND ENDFUNCTION co = COROUTINE.START(@Patrol)
```

Block form auto-starts. Great for "walk to A, wait, say line, walk to B" without state-machine spaghetti — until you make spaghetti anyway.

Chapter 10 — INCLUDE, IMPORT, and splitting your brain

```
INCLUDE "helpers.mb" INCLUDE "player.mb" IMPORT "physics_helper"
```

INCLUDE facts (from docs/reference/INCLUDE.md)

- Compile-time merge of `.mb` files (not Markdown docs)
- Path relative to the **including** file, then optional `MOONBASIC_PATH`
- Include-once (duplicates skipped)
- Circular includes → compile error
- **Zero** per-frame cost — it's paste-with-rules, not a runtime `require`

IMPORT

Loads package entry files from package roots (`MOONBASIC_PKG` / configured roots). Use for reusable libs. Use `INCLUDE` for "files next to my game."

Practical split

```
main.mb ; loop + scene wiring player.mb ; movement + jump enemies.mb ; AI that
cheats ui.mb ; menus
```

When three people edit one 3,000-line `main.mb`, friendships die.

Chapter 11 — Style: how cool kids write moonBASIC

Canonical: `Namespace.Method` + chaining

```
cam = CAMERA.CREATE().pos(0, 10, 20).look(0, 0, 0).fov(60) cube =  
ENTITY.CREATECUBE(1, 1, 1).scale(1.4, 1.4, 1.4).pos(0, 1, 0).col(255, 150, 70)
```

House rules:

1. **CREATE** not deprecated **MAKE**
2. **SETPOS** / **.pos** not dusty **SETPOSITION**
3. Chain setters that return self
4. **.free()** / ***.FREE** when done
5. Showcase path: **handle methods** — `hero.SetGravity(1.0)` over cafeteria-shouting
`CHARACTERREF.SETGRAVITY(hero, 1.0)`

Easy Mode is a jacket, not a personality

```
cam = CreateCamera() PositionEntity(cam, 0, 5, -10) ; preferred for new work:  
cam = CAMERA.CREATE().pos(0, 5, -10)
```

Full style bible: `STYLE_GUIDE.md`. Constitution: `API_STANDARDIZATION_DIRECTIVE.md`.

Part III — The Sacred Game Loop

Chapter 12 — Open a window or go home

setup once → WHILE running → update → clear → draw → FRAME → WEND → cleanup

Annotated foundation (examples/guides/game_loop.mb)

```
APP.OPEN(1280, 720, "Game loop demo") APP.SETFPS(60) cam = CAMERA.CREATE()  
CAMERA.SETPOS(cam, 0, 2, -8) CAMERA.LOOKAT(cam, 0, 0, 0) cube =  
ENTITY.CREATECUBE(2, 2, 2) cube.pos(0, 0, 5) WHILE NOT APP.SHOULDCLOSE()  
cube.turn(0, 60 * APP.DELTA(), 0) RENDER.CLEAR(20, 20, 30) RENDER.BEGIN(cam)  
SCENE.DRAW() RENDER.END() RENDER.FRAME() WEND ENTITY.FREE(cube) APP.CLOSE()
```

Step	Why
APP.OPEN / WINDOW.OPEN	Creates OS window + gfx context
SETFPS	Stable timing
Create world	Cameras, entities, lights
WHILE NOT ...SHOULDCLOSE	Until quit
Update with DELTA	Frame-independent motion
RENDER.CLEAR	Wipe previous frame
Begin / draw / end	3D (or 2D) pass
RENDER.FRAME()	Present. Skip this → blank/frozen hell.
Free + CLOSE	Don't haunt the next run

APP.* often aliases WINDOW.* / TIME.*. Pick a dialect per project and stick to it.

There is **no** hidden Game.Loop(). The WHILE is the product.

Chapter 13 — Input: keys, mouse, pads, ACTIONS

Raw keys

```
IF INPUT.KEYDOWN(KEY_W) THEN ... IF INPUT.KEYPRESSED(KEY_SPACE) THEN ... ;  
Easy: KeyHit / INPUT.KEYHIT on some paths
```

Axes & movement helpers

```
fwd = INPUT.AXIS(KEY_S, KEY_W) side = INPUT.AXIS(KEY_A, KEY_D) move =  
INPUT.MOVEMENT2D(KEY_W, KEY_S, KEY_A, KEY_D) ; [forward, strafe] style  
handle/tuple dir = INPUT.MOVEDIR(camYaw) ; camera-relative XZ
```

ACTION maps (remap without rewriting IF trees)

```
ACTION.MAPKEY("Jump", KEY_SPACE) ACTION.MAPKEY("Forward", KEY_W)  
ACTION.BINDGAMEPAD("Jump", PAD_A) IF ACTION.DOWN("Forward") THEN ; move ENDIF  
IF ACTION.HIT("Jump") THEN ; jump once ENDIF
```

Also: `INPUT.MAPKEY` / `INPUT.ACTIONDOWN` aliases depending on doc era — same idea.

Mouse

```
mx = INPUT.MOUSEX() my = INPUT.MOUSEY() IF INPUT.MOUSEPRESSED(MOUSE_LEFT) THEN  
... dx = INPUT.MOUSEDX() ; or MOUSEDX dy = INPUT.MOUSEDY()
```

Cursor lock (FPS vibes)

```
CURSOR.HIDE() CURSOR.DISABLE() ; relative / lock style ; or  
INPUT.LOCKMOUSE(TRUE) ; ... read deltas each frame ... CURSOR.SHOW()  
CURSOR.ENABLE()
```

Star Road (`main_easymode.mb`) hides the cursor at start and shows it on exit. Copy that politeness.

Gamepad

```
IF INPUT.GAMEPADCONNECTED(0) THEN lx = INPUT.GAMEPADAXIS(0, PAD_LEFT_X) ENDIF
```

See `examples/gamepad/main.mb`. Mapping save/load exists for people who bind jump to "mic mute" by accident.

Guide: `docs/systems/04-INPUT.md`, `CAMERA-AND-INPUT.md`.

Chapter 14 — Delta time, cameras 2D, and not teleporting

Delta or die

```
dt = APP.DELTA() ; aka TIME.DELTA() x = x + speed * dt
```

Without dt, 144 Hz players become speedrunners and 30 Hz laptops become molasses.

CAMERA2D

```
cam2d = CAMERA2D.CREATE() CAMERA2D.SETTARGET(cam2d, playerX, playerY)
CAMERA2D.SETZOOM(cam2d, 2.0) CAMERA2D.SETROTATION(cam2d, 0)
CAMERA2D.BEGIN(cam2d) ; world draw: tilemap, sprites in world space
CAMERA2D.END() ; HUD in screen space AFTER End DRAW.TEXT($"HP {hp}", 12, 12,
18, 255, 255, 255, 255)
```

Also: SETOFFSET, identity begin/end for pure screen space.

Chapter 15 — DRAW.* — rectangles, text, HUD glory

Default font via `DRAW.TEXT` — **no TTF required** for demos. Custom fonts: `FONT.LOAD` + font-aware draw APIs.

Greatest hits

```
RENDER.CLEAR(14, 22, 33) DRAW.RECTANGLE(0, 500, 800, 100, 40, 50, 60, 255)
DRAW.RECTLINES(10, 10, 100, 40, 255, 255, 255, 255) DRAW.CIRCLE(400, 300, 20,
255, 180, 80, 255) DRAW.LINE(0, 0, 100, 100, 255, 0, 0, 255)
DRAW.TEXT("HEALTH", 15, 12, 16, 255, 255, 255, 255) DRAW.TEXTURE(tex, INT(px),
INT(py), 255, 255, 255, 255) DRAW.PIXEL(x, y, r, g, b, a) DRAW.GRID2D(...) ;
when available – handy for layout RENDER.DRAWFPS(10, 10) RENDER.FRAME()
```

`DEBUG.PRINT` style HUD stacks exist for quick overlays — see `DRAW2D` / `DEBUG` refs.

Pong uses nothing but rectangles + text and still starts arguments at parties.

Chapter 16 — Your first 2D game: Pong energy

Real sample: `examples/pong/main.mb`

```
moonrun examples/pong/main.mb
```

What it teaches

1. Input → integrate with `dt` → collide → score → draw → `FRAME`
2. `MATH.CLAMP` keeps paddles from fleeing to the astral plane

- 3. HUD is just text
- 4. Two humans + one ball = infinite drama

Skeleton (condensed)

```
APP.OPEN(960, 540, "moonBASIC Pong") APP.SETFPS(60) ; py1, py2, bx, by, bvx,
bvy, scores... WHILE NOT (INPUT.KEYDOWN(KEY_ESCAPE) OR APP.SHOULDCLOSE()) dt =
APP.DELTA() ; W/S and I/K paddles ; bounce ball; score on miss
RENDER.CLEAR(12, 14, 22) DRAW.RECTANGLE(...) DRAW.TEXT($"P1 {p1s} P2 {p2s}",
...) RENDER.FRAME() WEND
```

After Pong

Next	Path
Gravity spite	examples/platformer/
Levels as data	examples/tilemap/
GUI pause menu	examples/gui_*



Part IV — Into the Third Dimension

Chapter 17 — Cameras, cubes, and existential dread

Spinning cube rite (examples/spin_cube/main.mb)

```
SetMSAA(0) APP.OPEN(800, 600, "moonBASIC – spinning cube") APP.SETFPS(60) cam
= CAMERA.CREATE().fov(55) cube = ENTITY.CREATECUBE(1, 1, 1).scale(1.4, 1.4,
1.4).pos(0.0, 1.0, 0.0).col(255, 150, 70) WHILE NOT (INPUT.KEYDOWN(KEY_ESCAPE)
OR APP.SHOULDCLOSE()) RENDER.CLEAR(38, 42, 58) dt = APP.DELTA()
cube.rot(...spin...) CAMERA.SETORBIT(cam, 0.0, 1.0, 0.0, camYaw, camPitch,
cdist) cam.Begin() Draw3D.Grid(14, 1.0) ENTITY.DRAWALL() cam.End()
Draw.Text("Spinning cube ESC quit", 12, 10, 18, 235, 240, 255, 255)
RENDER.FRAME() WEND cube.free() cam.free() APP.CLOSE()
```

Camera API buffet (learn as needed)

Trick	APIs
Look-at demo	SETPOS, LOOKAT / .look
Orbit third person	SETORBIT, ORBIT, ORBITENTITY, USEMOUSEORBIT
Follow	FOLLOW, LOOKATENTITY, LERPTO, SMOOTHEXP
FPS	SETFPSMODE, UPDATEFPS, LOCKMOUSE, mouse deltas
Juice	SHAKE
Picking	GETRAY, MOUSERAY, RAYCASTMOUSE, PICK, WORLDTOSCREEN

```
CAMERA.ORBIT(cam, hero, 8.0) CAMERA.USEMOUSEORBIT(cam, true)
CAMERA.FOLLOW(cam, hero, 0, 3, -8) INPUT.LOCKMOUSE(true)
```

Degrees vs radians: Star Road literally ships `CONST RAD_TO_DEG = 57.2957795` because mouse deltas are radians-ish and `SIN/COS` want degrees. Read comments in samples. Math is a prank.

Chapter 18 — Entities: your toys with handles

`ENTITY` is the big 3D toolbox (~hundreds of overloads — largest neighborhood in the audit).

```
cube = ENTITY.CREATECUBE(2, 2, 2) cube.pos(0, 1, 0) cube.col(255, 80, 80) cube
= ENTITY.CREATECUBE(1, 1, 1).scale(2, 2, 2).pos(0, 1, 0).col(80, 200, 255)
```

Universal vibes across spatial handles:

- `.pos` / `.rot` / `.scale` (get/set patterns)
- `.free()`
- Color / alpha helpers on many types

Drawing:

```
ENTITY.DRAWALL() ; or SCENE.DRAW() depending on sample era
```

Physics hooks:

```
ENTITY.PHYSICS(floor, "BOX", 0.0, 0.9, 0.0) ENTITY.ADDPHYSICS(ent) ; paths
vary by sample ENTITY.LINKPHYSBUFFER(...) ; advanced sync
```

Free your crap. Scene reset: `ENTITY.CLEARSCENE, ERASE ALL` (with the caveats from ch. 7/38).

Guides: `ENTITY-SYSTEM.md`, `reference/ENTITY.md`.

Chapter 19 — DRAW3D, grids, and debug geometry

Inside a 3D begin/end:

```
RENDER.BEGIN3D(cam) DRAW3D.GRID(20, 1.0) DRAW3D.CUBE(0, 0.5, 0, 1, 1, 1, 100,
180, 255, 255) DRAW3D.SPHERE(2, 1, 0, 0.5, 255, 100, 100, 255) DRAW3D.LINE(0,
0, 0, 0, 3, 0, 255, 255, 0, 255) DRAW3D.RAY(...) DRAW3D.BILLBOARD(...)
RENDER.END3D()
```

Short aliases in some builds: `BOX`, `BALL`, `GRID3`, `FLAT`, `CAP` — great for jams, easy to confuse with entity ownership. Prefer entities when the thing must persist, collide, or wear a material.

Debug lines: `DEBUG.DRAWLINE(...)` for "where the hell is my raycast."

Chapter 20 — Models, meshes, materials, textures

From `MESHES-MODELS-MATERIALS.md`:

```
tex = TEXTURE.LOAD("assets/brick.png") mat = MATERIAL.CREATE()  
MATERIAL.SETTEXTURE(mat, tex) MATERIAL.SETCOLOR(mat, 200, 200, 200) wall =  
ENTITY.CREATECUBE(4, 3, 0.2) ENTITY.SETMATERIAL(wall, mat) heroModel =  
MODEL.LOAD("assets/hero.glb") hero = ENTITY.CREATE("Hero")  
ENTITY.SETMODEL(hero, heroModel)
```

Also: `MESH.CUBE` / `SPHERE` / `PLANE`, animation queries (`MODEL.ANIMCOUNT`, `GETANIMNAME`), pack loading via `ASSET.*`.

Free with `TEXTURE.FREE` / `MODEL.FREE` / entity free. Unload animations before yeeting models in long sessions.

Set roots:

```
ASSET.PATH("assets/") model = MODEL.LOAD("player.glb")
```

Chapter 21 — Lights, fog, and "why is everything black"

Black scene checklist:

1. Clear color + no lights + dark materials = coal mine
2. Camera inside mesh / under floor
3. Forgot `ENTITY.DRAWALL` / `SCENE.DRAW`
4. Drawing outside Begin/End
5. Fog density set to "milk"

Fog (Star Road style)

```
WORLD.FOGMODE(1) ; linear WORLD.FOGCOLOR(55, 65, 120) WORLD.FOGDENSITY(0.05)
```

Lights

Explore `LIGHT.*` / `LIGHT2D.*`. Start with bright vertex colors on primitives before you invent a studio lighting thesis.

Post (optional flex)

Showcase toggles bloom / tonemap via `POST.ADD("bloom")`, `POST.SETTONEMAP(3)` (ACES-ish). Cool for trailers. Not required for fun.

Guides: `LIGHTING.md`, `examples/guides/lighting.mb`.

Part V — Physics: Newton's Revenge

Chapter 22 — Jolt, gravity, and respectful bouncing

Desktop **Windows/Linux** + **CGO** full runtime → real Jolt. Stubs elsewhere still **compile** the same `.mb` (Same Path), but balls may not bounce like your dreams.

Start + step

```
PHYSICS3D.START() PHYSICS3D.SETGRAVITY(0, -9.81, 0) ; showcase sometimes:  
PHYSICS3D.SETTIMESTEP(90.0) ; each frame: PHYSICS3D.UPDATE() ; or burrito:  
UPDATEPHYSICS() ; entity update + physics step convenience
```

Bodies

```
ballDef = BODY3D.CREATE("DYNAMIC") BODY3D.ADDSPHERE(ballDef, 1.0) ball =  
BODY3D.COMMIT(ballDef, 0, 10, 0)
```

Entity macros:

```
ENTITY.PHYSICS(floor, "BOX", 0.0, 0.9, 0.0)
```

Static shape path (Star Road):

```
floorShape = SHAPE.CREATEBOX(15, 0.5, 15) floorBody =  
Static.Create(floorShape) floorBody.SetPos(0, 0, 0)
```

Bounce / friction must reach Jolt

Setting only a visual field while the physics buffer keeps old restitution = flea-hop forever. Use APIs that update the body (`ENTITY.SETBOUNCE`, `ENTITY.SETFRICTION`, physics setters). See `PHYSICS.md` for dual-path sync, visual Y snap, and tiny vertical velocity kill when grounded.

Try: `examples/sphere_drop/main.mb`.

Chapter 23 — Characters that walk like humans (mostly)

KCC > ragdoll potato for platformers.

Hop sample (`modern_blitz_hop.mb`)

```
PHYSICS3D.START() WORLD.Gravity(0, -40, 0) hero = MODEL.CREATECAPSULE(0.4,
1.0) hero.Pos(0, 5, 0) CHAR.CREATE(hero, 0.4, 1.0) CHAR.SETSTEP(hero, 0.3)
CHAR.SETSLOPE(hero, 45.0) CHAR.STICK(hero, 0.5) floor = MODEL.CREATEBOX(100,
2, 100) ENTITY.PHYSICS(floor, "BOX", 0.0, 0.9, 0.0) cam.Orbit(hero, 12.0) ;
loop: CHAR.MOVEWITHCAMERA(hero, cam, fwd, side, 10.0) IF KEYPRESSED(KEY_SPACE)
AND Player.GetGrounded() THEN CHAR.JUMP(hero, 12.0) hero.Squash(0.5, 0.3)
ENDIF UPDATEPHYSICS()
```

Handle-method premium path

```
hero = playerEnt.CreateCharacter(0.5, 2.0) hero.SetPos(0, 5, 0)
hero.SetGravity(1.0) hero.SetStepHeight(0.4) hero.SetMaxSlope(45.0)
hero.SetSnapDistance(0.5) IF Player.GetGrounded() AND INPUT.KEYHIT(KEY_SPACE)
THEN hero.Jump(JUMP_FORCE) ENDIF
```

Refs: `CHARACTER.md`, `CHARACTER-3D-WALKING.md`, more under `examples/kcc_*.mb`.

Chapter 24 — Collision without a full physics world

2D — three paths (`COLLISION-2D.md`)

A. Manual rects (Pong / simple platformer):

```
IF py >= 398 THEN py = 400 pvy = 0 onGround = 1 ENDIF
```

B. Stateless helpers:

```
IF COLLISION.BOXOVERLAP2D(pa, sa, bp, bs) THEN ... ; CIRCLEOVERLAP2D,
POINTINBOX2D, CIRCLEBOX2D, LINESEGINTERSECT2D
```

C. Box2D — next chapter.

3D helpers

```
IF COLLISION.SPHEREOVERLAP3D(...) THEN ... IF COLLISION.BOXOVERLAP3D(...) THEN
... PICK.SCREENCAST(cam) IF PICK.HIT() THEN hitEnt = PICK.ENTITY()
```

Use math overlap for triggers/pickups; use Jolt when you need stacking crates and regret.

Chapter 25 — 2D physics with Box2D

```
PHYSICS2D.START() PHYSICS2D.SETGRAVITY(0, 500) def = BODY2D.CREATE("dynamic")
BODY2D.ADDRECT(def, 28, 28) player = BODY2D.COMMIT(def, 120, 360) ; loop:
PHYSICS2D.STEP() px = BODY2D.X(player) py = BODY2D.Y(player)
```

Tune with iterations / step settings for stability vs cost (PROGRAMMING.md perf notes). Guide:
PHYSICS-2D-PLATFORMER.md.

Part VI — Make an Actual Game

Chapter 26 — Platformers, hoppers, Mario-ish vibes

A game is:

1. Loop that never forgets `FRAME`
2. A verb (jump, shoot, talk)
3. Feedback (sound, squash, shake)
4. Failure
5. A reason to retry

Third-person hop recipe

1. `PHYSICS3D.START`
2. Static floor physics
3. Capsule + `CHAR.CREATE` / `CreateCharacter`
4. `cam.Orbit` / mouse look
5. Move in camera space
6. Jump when grounded
7. `UPDATEPHYSICS`
8. Draw + HUD + juice

Design tips

- One verb first
- Snappy camera > cinematic nausea
- Juice early (squash on jump is free dopamine)
- Kill micro-bounce via bounce/friction + grounding, not by rewriting gravity 40 times

Folder of truth: `examples/mario64/`.

Chapter 27 — Annotated tour: Star Road showcase

File: `examples/mario64/main_easymode.mb`

This is the "premium Easy Mode + handles" flex. Steal ideas, not necessarily every API spelling.

What it stacks

System	Trick
Cursor	<code>CURSOR.HIDE</code> / <code>SHOW</code>
Physics	<code>PHYSICS3D.SETTIMESTEP(90)</code> + <code>START</code>
Atmosphere	Fog mode/color/density
Nav	<code>NAV.CREATE</code> → grid → <code>Build</code> + <code>NAVAGENT</code>
Player	Capsule → <code>CreateCharacter</code> → gravity/slope/snap
Particles	<code>CREATEEMITTER</code> trail + bursts on stars
Platforms	<code>TYPE Platform</code> + <code>DIM</code> + <code>SHAPE/Static.Create</code>
Camera	Manual yaw/pitch from mouse deltas + <code>cam.orbit</code>
Collectibles	<code>DIST3D</code> , hide star, score++, burst
Cinema	F3 post stack bloom/tonemap
Debug	FPS graph, F1 heap dump, F4 nav debug

Movement sketch

```
hero = playerEnt.CreateCharacter(0.5, 2.0) hero.SetGravity(1.0) ; ... IF
Player.GetGrounded() AND INPUT.KEYHIT(KEY_SPACE) THEN hero.Jump(JUMP_FORCE)
ENDIF cam.orbit(px0, py0 + 0.5, pz0, camYaw, camPitch, CAM_DIST)
```

Why this chapter exists

Because tutorials that only show one cube lie to you about how systems compose. Star Road is composition with taste (and comments about radians).

Run:

```
moonbasic --check examples/mario64/main_easymode.mb moonrun
examples/mario64/main_easymode.mb
```

Chapter 28 — Sprites, tilemaps, and the 2D renaissance

Guide: `SPRITES-TILEMAPS-2D.md`. Sample: `examples/tilemap/`.

Sprite loop

```
hero = SPRITE.LOAD("assets/hero.png") WHILE NOT APP.SHOULDCLOSE()  
  SPRITE.SETPOSITION(hero, px, py) RENDER.CLEAR(40, 44, 52) SPRITE.DRAW(hero)  
  RENDER.FRAME() WEND SPRITE.FREE(hero)
```

Jam hack: `SPRITE.BUILTIN("player")` style placeholders when art isn't ready (when available in your build).

Tilemap loop

```
map = TILEMAP.LOAD("levels/arena.tmx") ; each frame: TILEMAP.DRAW(map) ; solid  
checks via TILEMAP.GETTILE / collision helpers ; on level change:  
TILEMAP.FREE(map)
```

Draw order: map → entities/sprites → HUD.

2D is not baby mode. 2D is where design skill can't hide behind bloom.

Chapter 29 — Audio that doesn't ghost you

Guide: `AUDIO-FEEDBACK.md`. Prefer **AUDIO.***.

Need	Pattern
SFX	<code>LOADSOUND</code> → <code>PLAYSOUND</code> → <code>FREESOUND</code>
Music	<code>LOADMUSIC</code> → <code>PLAYMUSIC</code> → <code>UPDATEMUSIC</code> every frame → <code>STOPMUSIC</code>
3D	<code>LISTENERCAMERA(cam)</code> + positional play
Pack	<code>ASSET.LOADPACK</code> → <code>ASSET.SOUND("jump")</code>

```
jump = AUDIO.LOADSOUND("audio/jump.wav") IF ACTION.HIT("Jump") THEN  
  AUDIO.PLAYSOUND(jump) theme = AUDIO.LOADMUSIC("audio/theme.ogg")  
  AUDIO.PLAYMUSIC(theme) ; EACH FRAME: AUDIO.UPDATEMUSIC(theme)
```

How to silence yourself by accident

Mistake	Result
---------	--------

Never <code>UPDATEMUSIC</code>	Theme is a ghost
<code>LOADSOUND</code> every jump	Leak city
3D SFX, no listener	Wrong / silent pan
Path typo	Perfect quiet

Chapter 30 — GUI menus that don't softlock

Immediate-mode raygui wrappers. Widgets return state **every frame**.

Counter (examples/gui_counter/main.mb)

```
APP.OPEN(800, 520, "GUI counter") GUI.ENABLE() GUI.THEMEAPPLY("DARK") uiFont =
FONT.LOAD("examples/gui_counter/fonts/InterVariable.ttf") GUI.SETFONT(uiFont)
count = 0 WHILE NOT (INPUT.KEYDOWN(KEY_ESCAPE) OR APP.SHOULDCLOSE())
RENDER.CLEAR(28, 30, 38) IF GUI.WINDOWBOX(56, 48, 688, 360, "Counter") THEN
count = 0 IF GUI.BUTTON(72, 144, 168, 44, "+1") THEN count = count + 1 IF
GUI.BUTTON(256, 144, 168, 44, "Reset") THEN count = 0 GUI.LABEL(72, 212, 640,
40, "Count = " + STR(count)) RENDER.FRAME() WEND
```

Form junkfood

```
name = GUI.TEXTBOX(20, 48, 320, 28, name, 64, editName) volume =
GUI.SLIDER(20, 92, 360, 24, "Volume", "", volume, 0.0, 100.0) muted =
GUI.CHECKBOX(400, 92, 120, 24, "Mute", muted) tabClose = GUI.TABBAR(20, 152,
420, 28, "Stats;Audio;Debug", tabs)
```

Themes: `GUI.THEMEAPPLY("DARK" | "LIGHT" | "CYBER" | "CANDY" | "TERMINAL" | ...)` — see `examples/gui_theme/`.

Softlock prevention: always have ESC / window close / a Resume button that actually sets `paused = 0`. Menus without exits are how demos become hostage situations.

Chapter 31 — Particles and juice

Canonical emitter (examples/guides/particles.mb)

```
fx = PARTICLE.CREATE() PARTICLE.SETRATE(fx, 60) PARTICLE.SETLIFETIME(fx, 0.4,
1.2) PARTICLE.SETSPEED(fx, 1, 3) PARTICLE.SETCOLOR(fx, 255, 200, 80, 255)
PARTICLE.SETCOLOREND(fx, 255, 80, 20, 0) PARTICLE.SETPOS(fx, 0, 1, 0)
PARTICLE.PLAY(fx) ; loop: PARTICLE.UPDATE(fx, APP.DELTA()) ; inside 3D pass:
PARTICLE.DRAW(fx) PARTICLE.FREE(fx)
```

Also: `SETTEXTURE`, `SETGRAVITY`, `SETSPREAD`, `SETBURST`, `STOP`. Easy Mode aliases in Mario samples: `CREATEEMITTER`, `EMITPARTICLE`, etc.

Juice checklist

- ☐ Shake on big hits
- ☐ SFX on every verb
- ☐ Damage flash
- ☐ Jump/land squash
- ☐ Score popups
- ☐ Pause that unpauses

Juice is not "polish later." Juice is how a cube becomes a character.

Chapter 32 — Math helpers so you stop reinventingSQRT

Guides under `docs/systems/guides/math/`. Samples: `examples/guides/math/*.mb`.

VEC2

```
vel = VEC2.CREATE(dx, dy) nrm = VEC2.NORMALIZE(vel) px = px + VEC2.X(nrm) *  
speed * APP.DELTA() VEC2.FREE(nrm) VEC2.FREE(vel)
```

Helpers: `ROTATE`, `LERP`, `MOVE_TOWARD`, `PUSHOUT`, `LENGTH`, `DIST`, `DISTSQ`. Scalar overloads often return multi-values without a handle — prefer those when you can to avoid churn.

VEC3

`DOT`, `CROSS`, `REFLECT`, `PROJECT`, `NORMALIZE`, rotate/transform helpers.

```
dir = VEC3.NORMALIZE(dx, dy, dz) px = px + VEC3.X(dir) * speed * APP.DELTA()
```

Prefer scalars when hot

`MATH.DIST2D`, `MATH.HDISTSQ`, clamp/lerp/angle helpers — less handle traffic in tight loops. See also `LESS_MATH.md` for gameplay shortcuts (spawn rings, land snaps, etc.).

Chapter 33 — Saves, files, JSON

Guides: FILES-AND-JSON.md, SAVE-AND-PROGRESS.md. Samples:
examples/guides/files_json.mb, save_progress.mb.

JSON config

```
IF FILE.EXISTS("options.json") THEN rawJson = FILE.READTEXT("options.json")
doc = JSON.PARSESTRING(rawJson) volume = JSON.GETFLOAT(doc, "audio.volume")
JSON.FREE(doc) ENDIF doc = JSON.PARSESTRING("{\"audio\":{\"volume\":0}}")
JSON.SETFLOAT(doc, "audio.volume", volume) FILE.WRITETEXT("options.json",
JSON.STRINGIFY(doc)) JSON.FREE(doc)
```

Path syntax like "player.health" on get/set is a thing — see JSON reference.

SAVE table

```
IF FILE.EXISTS("save1.json") THEN SAVE.READ("save1.json") level =
SAVE.GET("level") SAVE.SET("best", score) SAVE.WRITE("hiscore.json")
```

Also: FILE.DELETE, DB / CSV when your game becomes a spreadsheet with trauma.

Don't store secrets in the zip. Future-you is watching.

Part VII — Bigger Than Your Bedroom

Chapter 34 — Assets pipeline and packs

```
ASSET.PATH("assets/") ASSET.LOADPACK("assets/assets.json") tex =  
ASSET.TEXTURE("player") mdl = ASSET.MODEL("hero") sfx = ASSET.SOUND("jump")
```

Example manifest shape:

```
{ "textures": { "player": "textures/player.png" }, "models": { "hero":  
"models/hero.glb" }, "sounds": { "jump": "audio/jump.wav" } }
```

Guides: `ASSETS-PIPELINE.md`, `PROJECT-WORKFLOW.md`.

Benefits: one load path, named lookups, fewer stringly-typed typos sprinkled through combat code.

Chapter 35 — Terrain, open worlds, atmosphere

When a floor box isn't enough: `TERRAIN.*`, streaming, scatter, `WATER`, `SKY`, `WEATHER`, `WORLD.*`.

Samples:

- `examples/terrain_chase/`
- `examples/terrain_colored/`
- `examples/terrain_async/`

Guide: `TERRAIN-OPEN-WORLD.md`.

Hard-knock advice

1. Prototype the verb on a flat box
2. Add terrain when jumping already feels good
3. Profile before inventing Engine 2 inside Engine 1
4. Fog + sky tint hide a lot of sins (use wisely)

Nav mesh (from Star Road):

```
nav = NAV.CREATE() nav.SetGrid(60, 60, 0.5, -15, -15) nav.Build() agent =  
NAVAGENT.CREATE(nav) agent.SetSpeed(3.0)
```

AI namespaces also include `STEER`, `BTREE` — for when enemies need hobbies.

Chapter 36 — Multiplayer: when friends become lag

Tutorial: `FIRST_MULTIPLAYER_GAME.md`. Guides: `MULTIPLAYER.md`.

Layers

Layer	Namespaces	Use when
High	<code>SERVER</code> , <code>CLIENT</code> , <code>RPC</code>	Gameplay messages
Mid	<code>NET</code> , packets	Custom protocols
Low	<code>ENET</code>	You enjoy pain
Social-ish	<code>LOBBY</code>	In-process lobby helpers — not Steam

Critical rule

You must `TICK` every frame. There is no magical background game thread babysitting the network while you nap.

Host / client sketch (`testdata/mp_host.mb` / `mp_client.mb`)

```
; Host
SERVER.START(27777, 8) WHILE done = 0
  SERVER.TICK(0.016)
WEND
SERVER.STOP()
FUNCTION PING(msg, peerH)
  done = 1
ENDFUNCTION
```

```
; Client
CLIENT.CONNECT("127.0.0.1", 27777)
CLIENT.ONCONNECT("ONCONNECTED")
FUNCTION ONCONNECTED()
  RPC.CALLSERVER("PING", "hello")
ENDFUNCTION
WHILE i < 2000
  CLIENT.TICK(0.016)
  i = i + 1
WEND
CLIENT.STOP()
```

Two terminals, same machine, firewall may still yell about ports. Start local before inviting Discord.

Brutal truths: pick sync model; don't trust clients with gold; latency is real; multiplayer is a second game bolted onto the first.

Chapter 37 — Easy Mode for Blitz dinosaurs

Full map: `EASY_MODE.md`. Migration: `MIGRATION.md`. Index: `BLITZ_COMMAND_INDEX.md`.

Easy Mode	Canonical
<code>Graphics(w,h,title)</code>	<code>WINDOW.OPEN / APP.OPEN</code>
<code>CreateCamera()</code>	<code>CAMERA.CREATE</code>
<code>KeyDown / KeyHit</code>	<code>INPUT.KEYDOWN / KEYPRESSED</code>
<code>UpdatePhysics()</code>	<code>UPDATEPHYSICS</code>
<code>PositionEntity</code>	<code>ENTITY.SETPOS</code>
<code>Millisecs()</code>	<code>TIME.MILLIS</code>
<code>CreateCamera2D</code>	<code>CAMERA2D.CREATE</code>

Policy: learn `Namespace.Method` + handles first. Use Easy Mode to port, not to hide forever. **No # \$? % suffixes.**

Chapter 38 — Memory, FREE, and not leaking the moon

Doc: `MEMORY.md`.

Three layers of "memory"

1. Go GC (engine internals — not your problem daily)
2. Native Raylib / Jolt allocations
3. VM handle table (textures, arrays, cameras, ...)

Habits

```
cube.free() cam.free() TEXTURE.FREE(tex) PARTICLE.FREE(fx) ERASE enemies ;
scene teardown / shutdown: ERASE ALL APP.CLOSE()
```

- Per-resource free during long sessions
- `FreeAll / ERASE ALL` as safety net at shutdown
- Entity numeric IDs need entity APIs, not only `ERASE ALL`
- Shared / borrowed textures: free the owner, not every view
- Don't free mid-expression while still using the handle

Leaks feel like "my game gets slower after 20 minutes." That's not vibes. That's you.

Part VIII — Shipping & Surviving

Chapter 39 — Debug like a detective

```
ASSERT(playerTex <> 0, "Failed to load player texture!") DEBUG.ASSERT(health  
>= 0, "Health went negative!") DEBUG.ENABLE() DEBUG.WATCH("fps", APP.GETFPS())  
DEBUG.WATCH("hp", hp) DEBUG.LOG("Spawn wave " + STR(wave)) DEBUG.DRAWLINE(0,  
0, 0, 5, 0, 0) HELP("ENTITY.SETPOS") HELP("CAMERA")
```

CLI:

```
moonbasic --check main.mb
```

Showcase hooks: `DEBUG.SHOWFPSGRAPH`, `DEBUG.DUMPHEAP`, `DEBUG.LISTCOMMANDS`.

Guide: `DEBUG-AND-TESTING.md`.

Greatest hits bugs

1. Forgot `RENDER.FRAME()`
2. `moonbasic` instead of `moonrun`
3. `NOT a OR b`
4. Shadowed namespace variable
5. Stub physics / no CGO expectations
6. Camera inside floor
7. HUD drawn with wrong depth / inside 3D pass unexpectedly
8. Music never `UPDATEMUSIC`
9. Asset path wrong because you launched from the wrong cwd

Chapter 40 — Performance without becoming a monk

From `PROGRAMMING.md` and lived trauma:

- Multiply motion by `DELTA`
- One physics step strategy — don't step three times "for smoothness" unless you know why
- Free handles; avoid per-frame `LOAD`
- Prefer scalar math helpers over allocate-normalize-free every entity
- Match iterations to need (`Physics2D.SetIterations` etc.)

- Don't draw the whole universe when a chunk system exists
- Profile before rewriting — feelings aren't metrics

Good enough at 60 FPS ships. Perfect at 0 FPS doesn't.

Chapter 41 — Ship it to other humans

Official: `GETTING_STARTED.md` § Ship your game.

Option A — players install runtime

Ship your `.mb` / `.mbc` + `assets/`. They install the **same-tag full runtime** (not compiler-only). Run:
`moonrun main.mb`.

Option B — zip the runtime with the game

Folder containing `moonrun` + game + assets. Double-click / `Play.bat`:

```
@echo off moonrun main.mb
```

`moonbasic package windows / linux` helps when available in your release.

Checklist

- ☐ Same release tag for engine + bytecode
- ☐ Assets paths work from the shipped cwd
- ☐ README with controls
- ☐ No secrets in the zip
- ☐ You actually tested the zip on a clean folder

Chapter 42 — Where to go next

Learning path

1. Window + text
2. Pong
3. Spin cube
4. Orbit hop

- 5. Juice pass
- 6. Content pass (tilemap / second level)
- 7. Only then: net / open world / shader rabbit holes

Official trails

Want	Go
Install	<code>BEGIN_HERE.md</code>
Language law	<code>LANGUAGE.md</code>
Programming	<code>PROGRAMMING.md</code>
Every command	<code>API_CONSISTENCY.md</code>
Topic index	<code>COMMANDS.md</code>
40 systems	<code>systems/README.md</code>
Guides	<code>systems/GUIDES.md</code>
Physics	<code>PHYSICS.md</code>
Status	<code>STATE_OF_THE_UNION.md</code>
Examples	<code>examples/README.md</code>

Burnout protocol

Ship tiny. Limit scope. When stuck >30 minutes: `--check`, simplify to a cube. Touch grass. The moon will wait.

Appendices

Appendix A — Mega cheat sheet

```
; --- Window / loop --- APP.OPEN(w, h, "title") APP.SETFPS(60) dt =
APP.DELTA() WHILE NOT (INPUT.KEYDOWN(KEY_ESCAPE) OR APP.SHOULDCLOSE())
RENDER.CLEAR(r, g, b) RENDER.FRAME() WEND APP.CLOSE() ; --- 2D ---
DRAW.RECTANGLE(x, y, w, h, r, g, b, a) DRAW.CIRCLE(x, y, rad, r, g, b, a)
DRAW.TEXT(msg, x, y, size, r, g, b, a) CAMERA2D.BEGIN(cam2d) ...
CAMERA2D.END() ; --- 3D --- cam = CAMERA.CREATE().fov(60) cube =
ENTITY.CREATECUBE(1, 1, 1).pos(0, 1, 0).col(255, 200, 50) cam.Begin()
DRAW3D.GRID(20, 1) ENTITY.DRAWALL() cam.End() ; --- Input ---
INPUT.KEYDOWN(KEY_W) INPUT.KEYPRESSED(KEY_SPACE) INPUT.AXIS(KEY_S, KEY_W)
ACTION.MAPKEY("Jump", KEY_SPACE) ACTION.HIT("Jump") CURSOR.HIDE() :
CURSOR.DISABLE() ; --- Physics --- PHYSICS3D.START() UPDATEPHYSICS()
ENTITY.PHYSICS(floor, "BOX", 0, 0.9, 0) hero = ent.CreateCharacter(0.5, 2.0)
hero.Jump(9) ; --- Audio --- sfx = AUDIO.LOADSOUND("a.wav")
AUDIO.PLAYSOUND(sfx) AUDIO.UPDATEMUSIC(theme) ; --- Particles --- fx =
PARTICLE.CREATE() PARTICLE.PLAY(fx) PARTICLE.UPDATE(fx, dt) PARTICLE.DRAW(fx)
; --- Data --- PRINT($"hp={hp}") FILE.WRITETEXT("o.json", JSON.STRINGIFY(doc))
SAVE.SET("best", score) : SAVE.WRITE("s.json") ; --- Debug --- ASSERT(ok <> 0,
"nope") DEBUG.WATCH("hp", hp) HELP("CAMERA.CREATE") ; --- Cleanup ---
cube.free() : cam.free() : ERASE ALL
```

```
moonbasic new MyGame moonrun main.mb moonbasic --check main.mb moonbasic
main.mb moonbasic install-vscode
```

Appendix B — Namespace phone book

Curated from `COMMAND_AUDIT.md` (~160 namespaces exist — this is the party list):

Namespace	One-liner
(global)	PRINT, math, strings, arrays, misc builtins
ACTION	Named input actions
AUDIO / SOUND / MUSIC	Playback
BODY2D / PHYSICS2D / BOX2D	2D physics
BODY3D / PHYSICS3D / JOLT / JOINT3D	3D physics
CAMERA / CAMERA2D	Views
CHAR / CHARACTER / CHARACTERREF / CHARCONTROLLER / PLAYER	Characters / KCC

COLLISION	Overlap tests
CURSOR	Mouse cursor / lock
DEBUG	Watches, asserts, overlays
DRAW / DRAW3D	Immediate mode drawing
ENTITY / SCENE	Scene objects
FILE / JSON / SAVE / DB / CSV / CONFIG	Data on disk
GUI / UI / FONT	Menus & text
INPUT / GAMEPAD / KEY / MOUSE	Devices
LIGHT / LIGHT2D	Lighting
MATERIAL / MESH / MODEL / TEXTURE / IMAGE	Art
MATH / VEC2 / VEC3 / QUAT / MAT4 / RAND	Math
NET / SERVER / CLIENT / RPC / ENET / LOBBY	Multiplayer
PARTICLE / PARTICLE2D / PARTICLE3D	Juice
PICK / RAY / RAY2D	Picking
RENDER / POST / SHADER / EFFECT	Frame & post
SPRITE* / TILEMAP / ATLAS	2D world
TERRAIN / WORLD / WATER / SKY / WEATHER	Outdoors
TIME / TIMER / STOPWATCH	Clocks
TWEEN / TRANSITION	Motion / fades
WINDOW / APP / GAME / SYSTEM	Platform
NAV / NAVAGENT / STEER / BTREE	AI
ASSET	Packs & roots
COROUTINE	Async-ish scripts

Appendix C — Common "why is it broken" list

Symptom	Likely cause
No window	<code>moonbasic</code> not <code>moonrun</code> ; compiler-only zip
Blank / frozen	Missing <code>RENDER.FRAME()</code>
Weird quit loop	<code>NOT a OR b</code>
Speed changes with FPS	Forgot <code>* DELTA()</code>
Black 3D	Camera / lights / not drawing
Physics meh	Stub/no CGO; never <code>UPDATE</code>
Unknown command	Typo; shadowed namespace
Silent music	No <code>UPDATEMUSIC</code>
Works only on your PC	Paths, cwd, GPU, tag mismatch
Slow over time	Leaking loads / never <code>FREE</code>

Appendix D — Map of the official docs

Doc	Why
<code>BEGIN_HERE.md</code>	Start
<code>GETTING_STARTED.md</code>	Install + ship
<code>FIRST_HOUR.md</code>	Beginner philosophy
<code>LANGUAGE.md</code>	Language law
<code>PROGRAMMING.md</code>	Structure + perf
<code>STYLE_GUIDE.md</code>	Taste
<code>EASY_MODE.md</code>	Blitz jacket
<code>API_STANDARDIZATION_DIRECTIVE.md</code>	Constitution
<code>API_CONSISTENCY.md</code>	Every command
<code>COMMAND_AUDIT.md</code>	Namespace map
<code>COMMANDS.md</code>	Topic index

PHYSICS.md	Jolt truth
MEMORY.md	Handles & FreeAll
systems/README.md	40 systems
systems/GUIDES.md	Quests
architecture/HAL_AND_RENDERING.md	Pixels escape plan
ARCHITECTURE.md	Compiler/VM
STATE_OF_THE_UNION.md	Current goals
examples/README.md	Runnable catalog

Appendix E — Glossary

Term	Meaning
.mb / .mbc	Source / bytecode
moonrun	Play the damn game
moonbasic	Check / compile / LSP / scaffold
Namespace	ENTITY, CAMERA, ...
Handle	Live engine object reference
Easy Mode	Blitz-style wrappers; secondary
fullruntime	Build tag with graphical engine
HAL	Null vs Raylib drivers
Jolt / Box2D	3D / 2D physics
KCC	Kinematic character controller
Same Path	One .mb on Win/Linux
Juice	Feedback that feels good
ACTION	Named remappable input
INCLUDE	Compile-time file merge

Appendix F — Sample catalog (steal these)

Path	Steal for
<code>examples/guides/game_loop.mb</code>	Minimal loop
<code>examples/pong/main.mb</code>	2D + input + HUD
<code>examples/platformer/main.mb</code>	2D jump spite
<code>examples/tilemap/</code>	Tiled levels
<code>examples/spin_cube/main.mb</code>	First 3D
<code>examples/sphere_drop/main.mb</code>	Jolt drop
<code>examples/mario64/modern_blitz_hop.mb</code>	Orbit + KCC intro
<code>examples/mario64/main_easymode.mb</code>	Showcase composition
<code>examples/mario64/main_entities.mb</code>	Entity graph / CI check
<code>examples/guides/particles.mb</code>	Emitters
<code>examples/guides/lighting.mb</code>	Lights
<code>examples/guides/files_json.mb</code>	JSON
<code>examples/guides/save_progress.mb</code>	Saves
<code>examples/guides/math/*</code>	Vectors & angles
<code>examples/gui_counter/</code>	Buttons
<code>examples/gui_form/</code>	Forms
<code>examples/gui_theme/</code>	Themes
<code>examples/gamepad/</code>	Pads
<code>examples/terrain_chase/</code>	Terrain chase
<code>examples/fps/</code>	Mouse look energy
<code>testdata/mp_host.mb + mp_client.mb</code>	Tiny RPC

Index: `examples/README.md`.

Appendix G — Key constants you'll actually use

Exact names live in the input/key references — these are the greatest hits demos use constantly:

Constant	Typical use
<code>KEY_ESCAPE</code>	Quit
<code>KEY_W</code> <code>KEY_A</code> <code>KEY_S</code> <code>KEY_D</code>	Move
<code>KEY_SPACE</code>	Jump / confirm
<code>KEY_LEFT</code> <code>KEY_RIGHT</code> <code>KEY_UP</code> <code>KEY_DOWN</code>	Menus / pong right paddle vibes
<code>KEY_ENTER</code>	Confirm
<code>KEY_F1</code> ... <code>KEY_F4</code>	Debug toggles in showcases
<code>MOUSE_LEFT</code> <code>MOUSE_RIGHT</code>	Click / orbit
<code>PAD_A</code> <code>PAD_B</code> <code>PAD_LEFT_X</code> ...	Gamepad

If a key name fails --check, open `HELP("INPUT")` or the `INPUT` reference — don't invent `KEY_JUMP` unless you mapped an `ACTION` called Jump.

Appendix H — "Add systems in this order" poster

From the spirit of `systems/00-START.md`:

```
1. Window + loop + RENDER.FRAME 2. DRAW text / rectangles (prove you're alive)
3. INPUT / ACTION 4. DELTA-based movement 5. Camera (2D or 3D) 6. Assets
(textures / models / sounds) 7. Collision or physics 8. Audio feedback 9. GUI
/ pause 10. SAVE 11. Particles / post / juice 12. AI / nav 13. Multiplayer
(seriously, last)
```

Skip ahead at your own risk. Skipping audio until the end is fine. Skipping `FRAME` is not.

Part IX — Deep Cuts (the "I want more" section)

Chapter 43 — Full walkthrough: build Pong in your head

We're going to narrate `examples/pong/main.mb` like a sports commentator who knows how to code.

Act 1 — Stage

```
APP.OPEN(960, 540, "moonBASIC Pong") APP.SETFPS(60)
```

960×540 is "retro HD." Not sacred. Change it. Live a little.

Act 2 — Cast

Paddle size, clamp bounds, ball start, velocities, scores — plain variables. No ECS. No `PaddleComponent`. Just numbers with jobs:

```
pw = 16 ph = 80 py1 = 200.0 py2 = 200.0 bx = 480.0 by = 270.0 bvx = 280.0 bvy  
= 140.0 p1s = 0 p2s = 0
```

Act 3 — Loop religion

Every frame:

1. Read `dt`
2. Move paddles (`W/S, I/K`) × `dt`, clamp
3. Integrate ball
4. Bounce on top/bottom
5. Paddle hit → reverse X, tweak Y from hit offset, maybe speed up
6. Miss → other player scores, reset ball
7. Clear, draw center dashed line, paddles, ball, score text
8. **RENDER.FRAME()**

Act 4 — Why the hit code looks spicy

When the ball hits a paddle, demos often:

- Flip `bvx`
- Nudge speed so rallies escalate
- Add spin from `(ballY - paddleCenter)` so edge hits aren't boring

That's game feel. That's not "advanced engine." That's arithmetic with attitude.

Homework

1. Add a win screen at 11 points
2. Add `AUDIO.PLAYSOUND` on paddle hit
3. Remap controls with `ACTION.MAPKEY`
4. Add a third "chaos ball" and regret it

Chapter 44 — Full walkthrough: manual 2D platformer

File: `examples/platformer/main.mb`

No Box2D. No tilemap. Just you, gravity, and rectangles. Perfect for learning *why* physics engines exist — and when you don't need one yet.

State

```
px = 120 py = 360 pvx = 0 pvy = 0 ong = 0
```

Forces of nature (homemade)

```
; accelerate left/right IF INPUT.KEYDOWN(KEY_A) THEN pvx = pvx - 520 * dt IF
INPUT.KEYDOWN(KEY_D) THEN pvx = pvx + 520 * dt pvx = pvx * 0.88 ; friction-ish
IF ong = 1 AND INPUT.KEYDOWN(KEY_SPACE) THEN pvy = -420 px = px + pvx * dt py
= py + pvy * dt pvy = pvy + 980 * dt ; gravity
```

Collision: the honest lie

```
ong = 0 IF py > 400 THEN py = 400 IF py >= 398 THEN pvy = 0 IF py >= 398 THEN
ong = 1 ; floating platform AABB (simplified) IF px > 200 AND px < 520 AND py
> 300 THEN py = 300 pvy = 0 ong = 1 ENDIF
```

This is **not** a general collider. It assumes you mostly fall onto platforms from above. Side-hitting a platform may feel cursed. That's the lesson: manual AABB is great until levels get clever — then `TILEMAP / PHYSICS2D / COLLISION.*` earn their keep.

Draw

Ground, two platforms, player rect, help text, `FRAME`. Done.

Upgrade path

1. Camera follow (CAMERA2D) when the level grows
2. TILEMAP.LOAD instead of hard-coded platforms
3. Swap to Box2D when you want slopes and moving platforms that aren't made of tears
4. Add coyote time + jump buffer (feel upgrades — still just timers)

Chapter 45 — Game recipes (copy-paste genres)

Use these as shopping lists, not prisons.

Recipe: Top-down crawler

- APP.OPEN + loop
- CAMERA2D follow
- TILEMAP or draw grid
- INPUT.MOVEMENT2D or WASD axes
- COLLISION.BOXOVERLAP2D for walls/enemies
- AUDIO.PLAYSOUND on hit
- SAVE for deepest floor

Recipe: Score attack arcade

- Spawner timer (STATIC or global $t = t + dt$)
- Object pool via DIM + active flags
- Escalating speed
- Particles on death
- Local hiscore via SAVE / JSON

Recipe: Third-person collectathon

- Star Road lite: KCC + orbit cam + DIST3D pickups + fog + one enemy NAVAGENT
- Don't start with nav mesh — start with 3 stars and a floor

Recipe: Racing (flat)

- Car = entity/model

- Acceleration along facing
- `cam.Follow` behind
- Lap trigger volumes (`COLLISION` or distance to points)
- See `examples/racing/` for inspiration

Recipe: FPS peek

- `CURSOR.DISABLE` + mouse deltas
- Camera yaw/pitch
- WASD on XZ basis (`INPUT.MOVEDIR` / camera basis helpers)
- Hitscan: `CAMERA.MOUSERAY` / `PICK`
- See `examples/fps/`

Recipe: Visual novel (yes, really)

- `GUI` or `DRAW.TEXT` + portraits as `DRAW.TEXTURE`
- `INCLUDE` script files or JSON dialogue
- Coroutines for "typewriter wait → next line"
- Save slot for route flags

moonBASIC isn't only cubes. Cubes are just honest.

Chapter 46 — Strings, printing, and talking to humans

```
PRINT "hello" PRINT("hello") COLORPRINT(100, 255, 100, "fancy console line")
name = "Ash" msg = "Hi " + name msg2 = $"Hi {name}, score={score}" msg3 =
STRING.INTERP("Hi {0}, score={1}", name, score) s = STR(42) n = VAL("3.14") u
= UPPER(name) f = FORMAT(yaw, "%.2f")
```

HUD:

```
DRAW.TEXT($"HP {hp}/{MAX_HP}", 12, 12, 18, 255, 220, 220, 255)
```

Console `PRINT` is for debug. Players look at the window. Don't put the plot exclusively in the terminal unless you're making *Terminal Quest III*.

String reference: `reference/STRING.md`.

Chapter 47 — A week-long curriculum

Day 1 — Alive

- Install IDE or full runtime
- `moonbasic new`
- Change clear color daily until it hurts
- Read Chapters 1–5

Day 2 — Language

- Variables, IF, FOR, FUNCTION
- Write a guessing game in the console (`PRINT` + `INPUT` if available, or keys)
- Chapters 6–11

Day 3 — 2D game

- Clone Pong ideas into your project
- Add score win state
- Chapters 12–16, 43

Day 4 — 3D

- Spin cube → orbit camera → your own colored platforms
- Chapters 17–21

Day 5 — Physics body

- Sphere drop or hop sample
- Tune gravity/jump until it feels like *your* game
- Chapters 22–27

Day 6 — Juice + data

- Sound on jump, particles on land, save hiscore
- Chapters 29–33

Day 7 — Ship a zip

- Package Option B from Chapter 41
- Give it to a friend with zero context
- Watch them press the wrong keys
- Fix the README
- Celebrate with snacks

Chapter 48 — FAQ from the trenches

Q: Why does nothing appear?

A: `RENDER.FRAME()` missing, or you ran `moonbasic` instead of `moonrun`.

Q: Why is my window instantly closed?

A: Loop condition wrong — check the `NOT (a OR b)` parentheses.

Q: Can I use Blitz # suffixes?

A: No. Plain names. Welcome to the future / present.

Q: APP vs WINDOW?

A: Often aliases. Pick one per project.

Q: CREATE vs MAKE?

A: `CREATE`. `MAKE` is deprecated comfort food.

Q: Why won't 3D physics bounce on my machine?

A: Need full runtime with Jolt (Windows/Linux + CGO builds). Stubs still compile.

Q: How do I pause?

A: `paused` flag — skip gameplay update when true; still draw + `FRAME`; GUI Resume sets flag false.

Q: How do I do scenes/levels?

A: Functions or INCLUDED files + `ERASE ALL` / entity clear between scenes. Or one big state enum: `MENU`, `PLAY`, `PAUSE`, `GAMEOVER`.

Q: Is moonBASIC object-oriented?

A: Handles + methods + namespaces. Not Java. Thank the moon.

Q: Can it make mobile games?

A: Desktop-first story (Windows then Linux). Don't assume phone packaging is a first-class button today — check current `STATE_OF_THE_UNION` / releases.

Q: Where is the entire command list?

A: `API_CONSISTENCY.md`. Bring coffee.

Q: My coworker writes `Window.Open` and I write `WINDOW.OPEN`.

A: Both fine. Case insensitive. Agree on a style guide before the duel.

Chapter 49 — For engine contributors (optional pain)

Only if you're hacking **this repo**, not just making games.

Goal	Command
Headless compiler	<code>go build -o moonbasic .</code>
Play from source	<code>go build -tags fullruntime -o moonrun ./cmd/moonrun</code>
Check sample	<code>go run . --check examples/mario64/main_entities.mb</code>
Tag matrix	<code>scripts/build/check_builds.ps1 / .sh</code>

Remember:

- Default `go run . file.mb` **compiles**, doesn't play
- Physics sync files use Windows+Linux tags — don't linux-only fork entity sync
- Manifest changes: edit `compiler/builtinmanifest/commands.json`, then `--check` samples; regen API docs with `go run ./tools/apidoc` when the public surface changes
- HAL: prefer `rt.Driver` for new window/input work; migration is incremental
- Read `DEVELOPER.md`, `CONTRIBUTING.md`, `PHYSICS.md`, `AGENTS.md`

Beginners: you can ignore this chapter forever and still ship bangers.

Chapter 50 — Porting from Blitz / DBPro without crying

1. **Delete suffix punctuation** from names (`# $ % ?`)
2. Map `Graphics` → `APP.OPEN` / `WINDOW.OPEN`
3. Map entity transforms → `ENTITY.SETPOS` / handle `.pos`
4. Prefer canonical namespaces; keep Easy Mode wrappers only where they save time
5. Replace `Include` habits with `moonBASIC INCLUDE "file.mb"`
6. Test with `--check` often — port in slices, not one 8,000-line prayer
7. Use `MIGRATION.md`, `BLITZ_COMMAND_INDEX.md`, `reference/dbpro/`

Emotional advice: your old game will not port in a weekend if it was a career. Port one room. Celebrate. Port the player. Celebrate harder.

Epilogue — Go make a mess

You don't need permission.

Open a window. Draw a rectangle. Spin a cube. Make a capsule jump on a green box until it feels good. Add a *bwip*. Show a friend. Watch them miss the jump and blame the game. Fix the jump. Ship a zip.

moonBASIC won't write your game for you — thank god — but it will get out of the way long enough for you to hear your own ideas.

Now stop reading books about making games and go make a stupid little game that only you love.

Then make another one.

— end of expanded book —

Written for humans. Validated against moonBASIC docs and samples in this repository. Occasional swearing included at no extra charge. If something here drifts from `commands.json`, the manifest wins — file a fix, don't start a cult.